Tribhuvan University

Faculty of Humanities and Social Sciences

PROJECT ARCHIVAL USING HUFFMAN ENCODING ALGORITHM

A PROJECT REPORT

Submitted To

Department of Computer Application

College Name

In partial fulfillment of the requirements for the Bachelors in Computer Application

Submitted By:

Pasang Gelbu Sherpa (1059xxxxx)

August 2025

6th Semester

Under the supervision of

Mr. Supervisor Name

Tribhuvan University

Faculty of Humanities and Social Science

College Name

SUPERVISOR'S RECOMMENDATION

I hereby recommend that this project prepared under my supervision by **PASANG GELBU SHERPA** entitled **"PROJECT ARCHIVAL USING HUFFMAN ENCODING ALGORITHM"** in partial fulfillment of the requirements for the degree of Bachelor of Computer Application is recommended for the final evaluation.

.....

SIGNATURE

Mr. Supervisor Name

Supervisor

College Name

College Full Address

Tribhuvan University

Faculty of Humanities and Social Science

College Name

LETTER OF APPROVAL

This is to certify that this project prepared by **PASANG GELBU SHERPA** entitled **"PROJECT ARCHIVAL USING HUFFMAN ENCODING ALGORITHM"** in partial fulfillment of the requirements for the degree of Bachelor's in Computer Application has been evaluated. In our opinion it is satisfactory in the scope and quality as a project for the required degree.

…… **Mr. Supervisor Name** **Supervisor** College Name College Full Address	…….. **Er. HOD Name** **Head of Department** College Name College Full Address
…….. **Internal Examiner**	…….. **External Examiner**

Abstract

With regard to academics and professions, managing and storing project files is often crucial. The Project Archival System solves problems with file storage by compressing and encrypting files through a web based system. The system allows the users, in this case, students, to upload project files or folders, compressing them into zip files to reduce storage amount. The system utilizes Huffman Algorithm and Zip Deflate Algorithm. Project Archival system offers a dynamic and responsive front-end, accomplished with a combination of Angular, which powers the front end, Spring Boot, which does the back-end processing, and a reliable PostgreSQL for data handling. The user can upload files, fetch project data, and fetch detailed analysis of the original versus the zip file. Additionally, the user can view insights into storage savings. The system enables efficient space and data management. Project Archival aids students organize and preserve project data with ease.

Keywords: *Huffman Algorithm, Zip Deflate Algorithm, Angular, Spring Boot, PostgreSQL*

Acknowledgements

I would like to express my sincere gratitude to everyone who contributed to the development of the "Project Archival" project. First and foremost, we would like to thank our project supervisor, **Mr. Supervisor Name** Sir for his invaluable guidance, support, and encouragement throughout this project. His expertise and insights were instrumental in shaping the direction and scope of this project.

Additionally, we extend our appreciation to our Coordinator **Er. Coordinator Name** for providing the resources and support necessary to complete this project. The access to facilities and technology greatly facilitated our work.

Finally, we are thankful to our families and friends for their unwavering support and encouragement throughout this journey. Their belief in our abilities kept us motivated and focused.

This project would not have been possible without the contributions of all these individuals and organizations. Thank you for your support.

Sincerely,

Pasang Gelbu Sherpa (1059xxxxx)

Table of Contents

List of Figures

List of Tables

List of Abbreviations

Abbreviation	Definition
API	Application Programming Interface
CASE	Computer-aided software engineering
DB	Database
DFDs	Data Flow Diagrams
ER	Entity Relationship
HTTPS	Hypertext Transfer Protocol Secure
IDE	Integrated Development Environment
JWT	JSON Web Token
OTP	One-Time Password
SPA	Single Page Application
TS	Typescript
UI	User Interface

Chapter 1: Introduction

1.1 Introduction

Project Archival is an innovative and user-friendly system designed to streamline the management, compression, and organization of student projects in an academic environment. This platform serves as a centralized repository where students can securely upload, compress, and store their project folders, while also gaining access to view other uploaded projects. With robust features for both students and administrators, Project Archival ensures efficient project management and seamless collaboration.

1.2 Problem Statement

- Manual project uploads and storage are disorganized, leading to difficulties in retrieving and managing files.
- There is no centralized platform for students to upload, compress, and view their projects, resulting in missed opportunities for collaboration and learning.
- Difficulty in managing the resources and getting the immediate access.

1.3 Objectives

- To compress file size for different document using Huffman Encoding Algorithm.
- To display the actual compression size and compare it with the previous versions to users.
- To integrate user based two factor authentication system.

1.4 Scope and Limitation

1.4.1 Scope

- **Project Upload and Storage:**
  Students can upload their project folders to the system, which are automatically compressed for efficient storage.
- **User Role and Access Control**:
  The system supports role-based access (e.g., students, super admins) to ensure secure and regulated usage.
- **Super Admin Management**:
  Super admins can manage project directories, organize files, and oversee user accounts (add, remove, or modify users).

1.4.2 Limitations

- Limited file support
- There can be the lack of proper security in the system.

1.5 Development Methodology

For the methodology of the proposed system, I am using the Waterfall methodology as the development model for our system as this model is very easy to understand than other models. It is easy to manage and arrange tasks. Each phase must be completed before the new phase starts, so there is no overlapping in the phases.

The following illustration is the representation of different phases of the waterfall model:

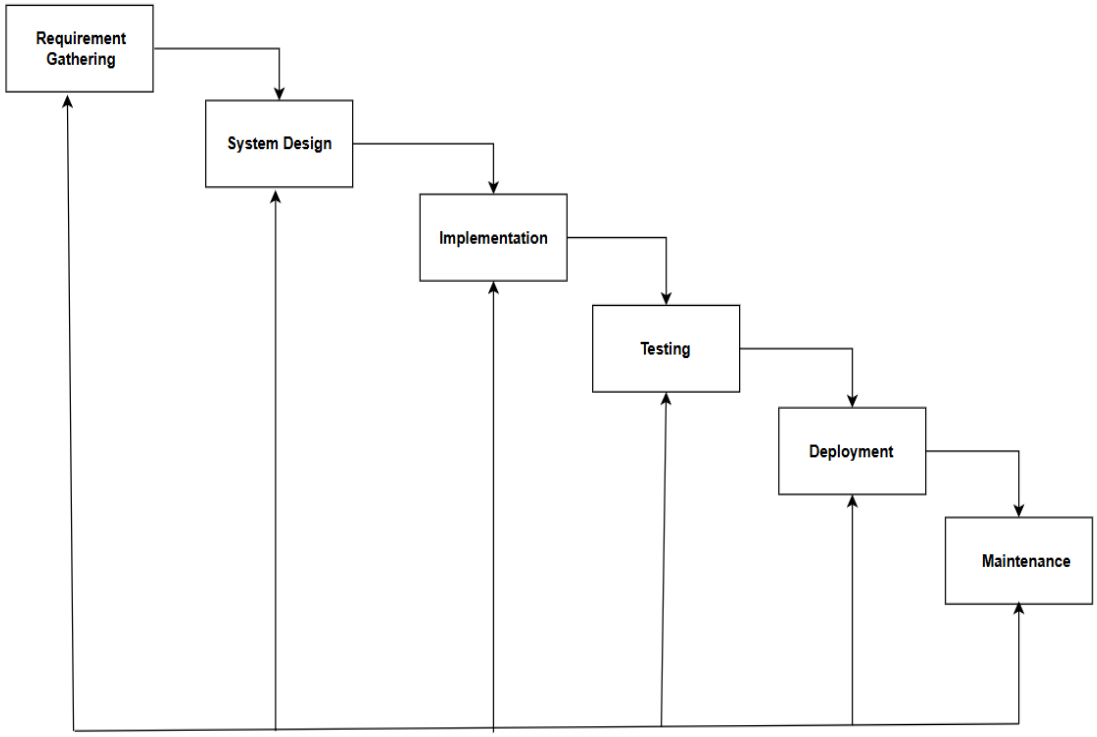


Figure 1. 1 Waterfall Model

1.6 Report Organization

Chapter 1: Introduction

This chapter presents an overview of the Project Archival project, outlining its importance in the context of academic planning and storage management. It defines the problems being addressed and explains the motivation behind the project. The chapter clearly states the objectives, scope and limitations of the system, providing a structured understanding of what the project intends to achieve. Finally, it concludes with a brief summary of the report structure.

Chapter 2: Background Study and Literature Review

The second chapter includes background study i.e., description of fundamental theories, general concepts and terminology related to the project. It also includes the literature review i.e., review of the similar projects, research and theories done by other researchers.

Chapter 3: System Analysis and Design

This chapter discusses the system requirements, both functional and non-functional, supported by use case diagrams to depict user interactions. It also includes data modeling through Entity Relationship (ER) diagrams and process modeling using Data Flow Diagrams (DFDs). The design section elaborates on the system architecture, interface design, database schema, and navigation flow, ensuring a clear understanding of how the system is structured.

Chapter 4: Implementation and Testing

This chapter covers the practical implementation of the Project Archival system using Angular for the frontend, Spring Boot for the backend, and PostgreSQL as the database. It describes the development of key modules including file and folder compression, User authentication. The chapter also details the testing processes applied, including unit testing, integration testing, and system testing, to ensure reliability and accuracy.

Chapter 5: Conclusion and Future Recommendations

The final chapter summarizes the project outcomes, lessons learned, and suggestions for future enhancement.

Chapter 2: Background Study and Literature Review

2.1 Background Study

File compression and archival are fundamental problems in computer science, involving the reduction of data size for efficient storage and retrieval. In the context of this project, Project Archival is a web-based application designed to allow students to upload, compress (using ZIP), and store project files in an optimized manner. The archival process can be conceptualized as a graph where each file represents a node, and compression operations represent edges that transform the file into a more compact form. The goal is to minimize storage space while preserving data integrity.

The ZIP compression process, central to Project Archival, relies on the Deflate algorithm, which combines LZ77 repetition elimination and Huffman encoding to achieve lossless compression [1]. The compression process proceeds as follows:

1. **Identify repetitions**: Apply LZ77 to detect and replace repeated sequences in the file with back pointers.
2. **Encode data**: Use Huffman encoding to assign variable-length codes to symbols based on their frequency, reducing the overall bit representation.
3. **Store compressed data**: Write the compressed data, including headers for Huffman trees, to enable decompression.

This process ensures efficient storage, making it suitable for academic environments where resources are limited and students need a simple, effective way to archive project files.

2.2 Literature Review

Several studies have explored compression techniques relevant to the Project Archival system, particularly those involving the Deflate algorithm and its components. The following works provide insights into the efficiency and applicability of these techniques:

Harnik et al. [1] present a fast implementation of the Deflate protocol, combining the LZ4 compressor with Zlib's Huffman encoding. Their approach achieves a compression speed 2.6 times faster than Zlib's fastest mode, with a negligible increase in compression ratio (40.92% vs. 40.37% on academic datasets). The study introduces a semi-static Huffman approach, where an initial block uses a static Huffman tree, and subsequent blocks leverage a predefined tree based on prior statistics. This method balances speed and compression efficiency, making it highly relevant for Project Archival's goal of rapid, storage-efficient archiving in resource-constrained settings.

Moffat provides a comprehensive overview of Huffman coding, a key component of the Deflate algorithm. The paper details the construction of minimum-redundancy prefix-free codes and discusses efficient algorithms, such as van Leeuwen's linear-time approach for computing codeword lengths. [2] It also highlights canonical Huffman codes, which enhance decoding efficiency by structuring codewords lexicographically. These techniques are critical for Project Archival, as they ensure that large project files can be compressed and decompressed efficiently, minimizing storage requirements while maintaining data integrity.

Deutsch defines the Deflate Compressed Data Format Specification, outlining the standard for combining LZ77 and Huffman coding. [3]. The specification emphasizes the importance of compatibility and efficiency in compression, which underpins the ZIP functionality in Project Archival. The standard's widespread adoption in tools like Zlib makes it a foundational reference for ensuring the system's interoperability with existing archival formats.

In summary, Project Archival leverages the Deflate algorithm, informed by studies such as [1], [2], and [3], to provide a user-friendly, storage-efficient solution for archiving academic project files. These works demonstrate the effectiveness of combining LZ77 and Huffman coding for compression, making them ideal for the project's focus on simplicity and efficiency in educational settings.

Chapter 3: System Analysis and Design

3.1 System Analysis

This chapter discusses about the proposed model of the system, the methodology used in building the system, tools and techniques, requirement analysis, requirement specification. It also talks about the functional requirements of the system, the system design. The application architecture of the system will also be shown in this chapter, the use case diagram, data design, activity diagrams, dataflow diagram, control flow diagram, entity-relationship diagram (ERD) and user interface design will all be shown in this chapter.

3.1.1 Requirement Analysis

The following section presents the complete set of functional requirements of Project Archival system.

i. Functional Requirements

Following figure shows the requirements of Project Archival system:

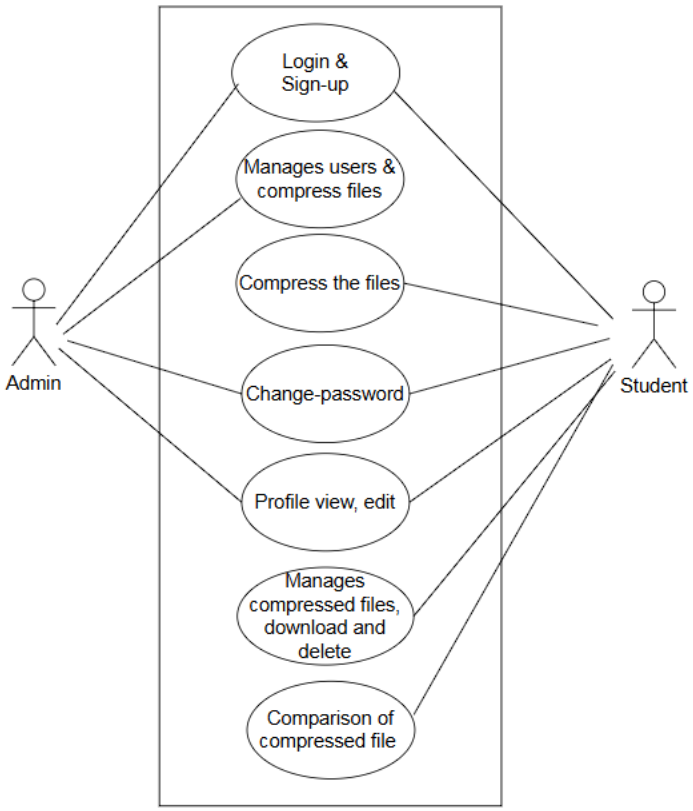


Figure 3. 1 Use Case Diagram of Project Archival

Use Case Description:

Table 3. 1 Use case Scenario for Register

Use-Case Identifier	UC1: Register
Primary Actor	User
Secondary Actor	X
Description	New users register by providing necessary information to access the system
Pre-Condition	User must not be already registered.
Success Scenario	User account is created successfully.
Failure Scenario	Registration fails due to invalid data or existing email.

Table 3. 2 Use Case Scenario for Login

Use-Case Identifier	UC2: Login
Primary Actor	User, Admin
Secondary Actor	X
Description	Users login with valid credentials to access the system.
Pre-Condition	User/Admin account must exist with valid credentials.
Success Scenario	User/Admin is logged in successfully.
Failure Scenario	Login fails due to invalid username/password.

Table 3. 3 Use Case scenario for Update Profile.

Use-Case Identifier	UC3: Update Profile (User)
Primary Actor	User
Secondary Actor	X
Description	User update their profile.
Pre-Condition	User must be logged in.
Success Scenario	Profile of user updated and viewed successfully.
Failure Scenario	Operation fails due to invalid input or system error.

Table 3. 4 Use Case Scenario for Compression

Use-Case Identifier	UC4: Compress File and Folders
Primary Actor	User
Secondary Actor	X
Description	User compress the files and folders.
Pre-Condition	User must be logged in.
Success Scenario	Files and Folders are compressed successfully.
Failure Scenario	Files or folder are not compressed and error message showed.

Table 3. 5 Use Case Scenario for Change Password

Use-Case Identifier	UC5: Change Password
Primary Actor	User, Admin
Secondary Actor	X
Description	User and Admin can change the password from old to new one.
Pre-Condition	They must be logged-in.
Success Scenario	Password Changed successfully.
Failure Scenario	If the password that need to be changed does not matched.

ii. Non Functional Requirements

1. **Performance Requirement**
   - Fast API response for file compression and data updates.
2. **Accessibility**
   - The system is web-based and accessible via standard web browsers such as chrome, Firefox, Microsoft etc.
   - User should not require additional plugins or installations to access the system.
3. **Security**
   - Secure login should be enforced for all registered users, using industry-standard authentication protocols.
   - Sensitive user data, including email, password, and compression data, must be securely encrypted both in transit (HTTPS) and at rest.
   - The system must implement role-based access control (e.g., registered users vs. administrators).

3.1.2 Feasibility Analysis

Some important feasibility studies are mentioned below:

i. Technical Feasibility

It is technically feasible as I already have hardware and software required for development of a software. Also, I have technical knowledge on how the project is made through programming language like JavaScript (JS), TypeScript (TS), Java, etc.

ii. Operational Feasibility

It is operationally feasible as I am making this system by removing the threats and weakness of existing non manageable system which is reliable for the users.

iii. Economic Feasibility

The system does not require extra software and hardware. So, there is no recurring cost than just the internet connection.

iv. Schedule Feasibility

The project is achievable within a 3–4 month timeline using a standard development model like Agile or Waterfall. Tasks can be divided efficiently across planning, development, and testing phases.

Task	Day Needed	Start Date	End Date
Request Analysis	10	2/23/2025	3/5/2025
System Design	14	3/6/2025	3/20/2025
Implementation	70	3/21/2025	5/30/2025
Testing	8	5/31/2025	6/8/2025
Deployment	8	6/9/2025	6/17/2025
Maintenance	5	6/18/2025	6/23/2025
Documentation	120	2/23/2025	6/23/2025

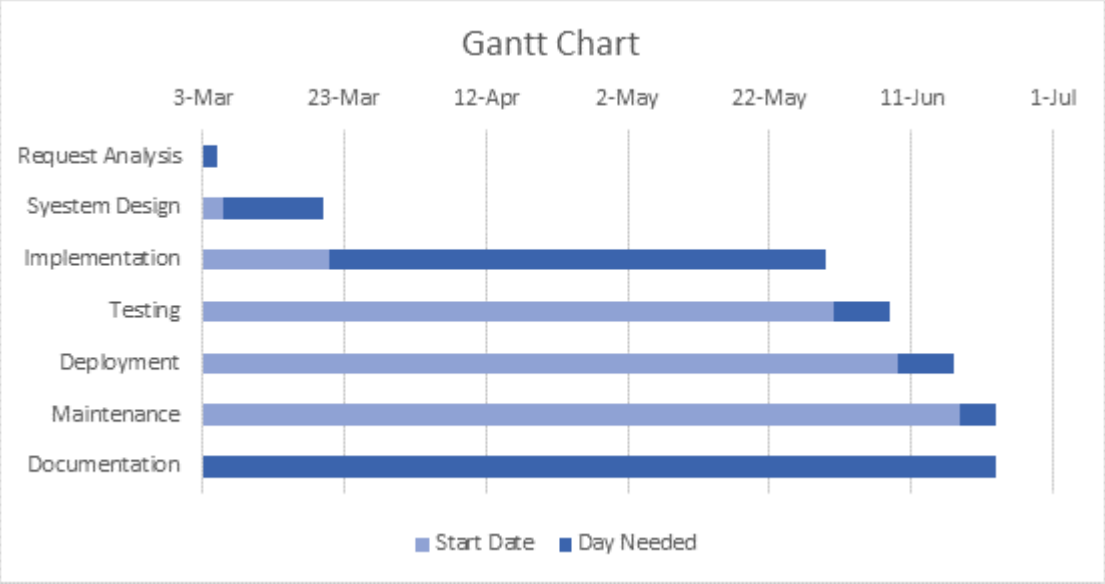


Figure 3. 2 Project Gantt Chart of Project Archival

3.1.3 Data Modeling (ER-Diagram)

The ER Diagram of Project Archival is shown below;

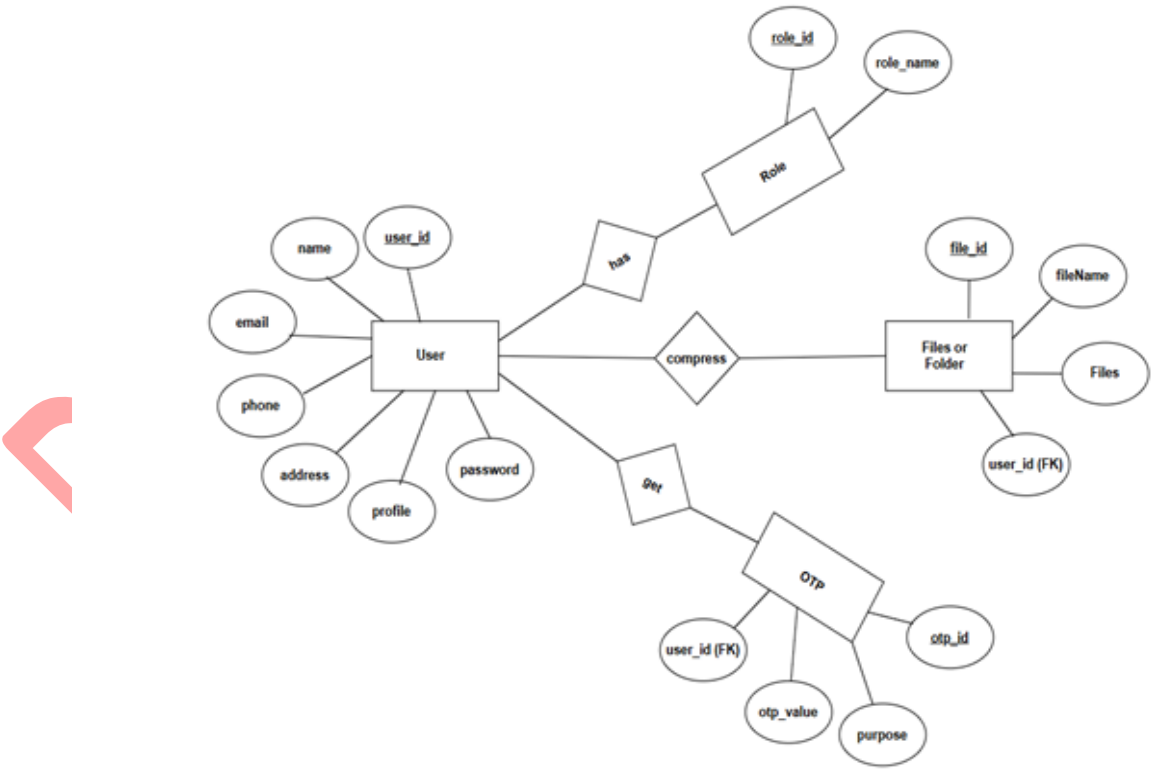


Figure 3. 3 Entity Relationship for Project Archival

3.1.4 Process Modeling (DFD)

The Level 0 and Level 1 Data flow diagram of the Project Archival are below:

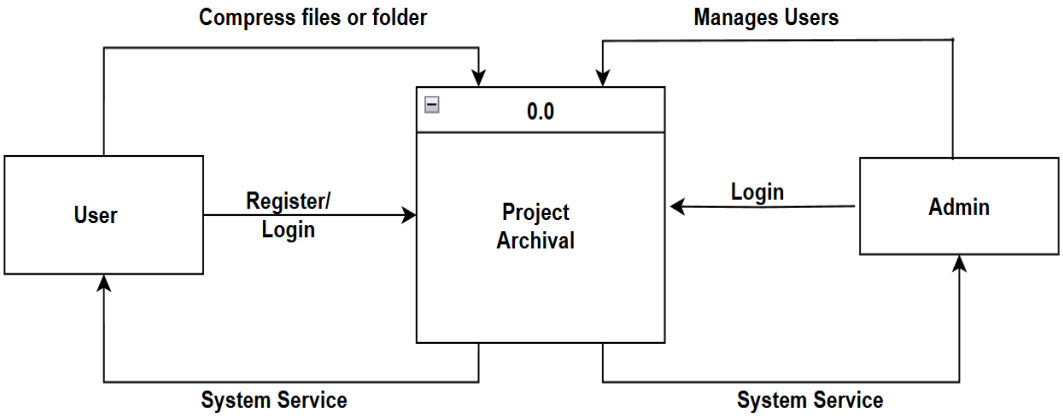


Figure 3. 4 Level 0 DFD of Project Archival

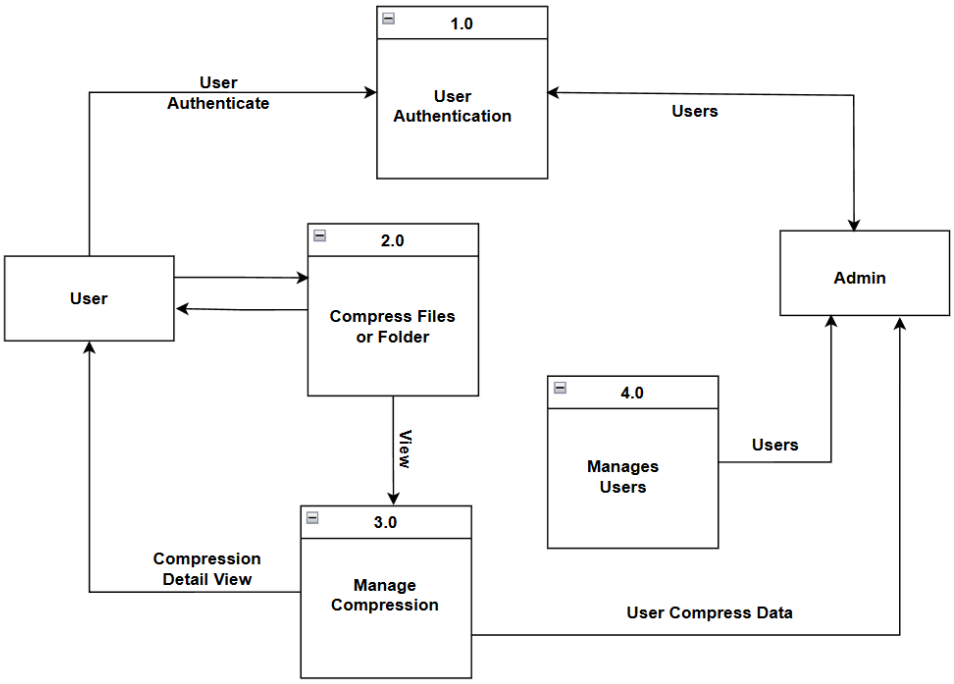


Figure 3. 5 Level 1 DFD of Project Archival

3.2 System Design

3.2.1 Architecture Design

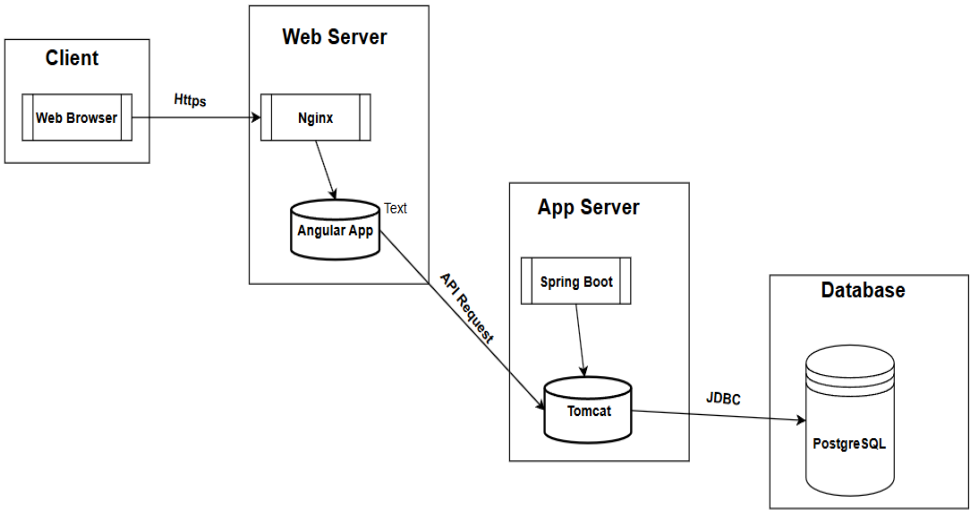


Figure 3. 6 Architecture Design of Project Archival

3.2.2 Database Schema Design

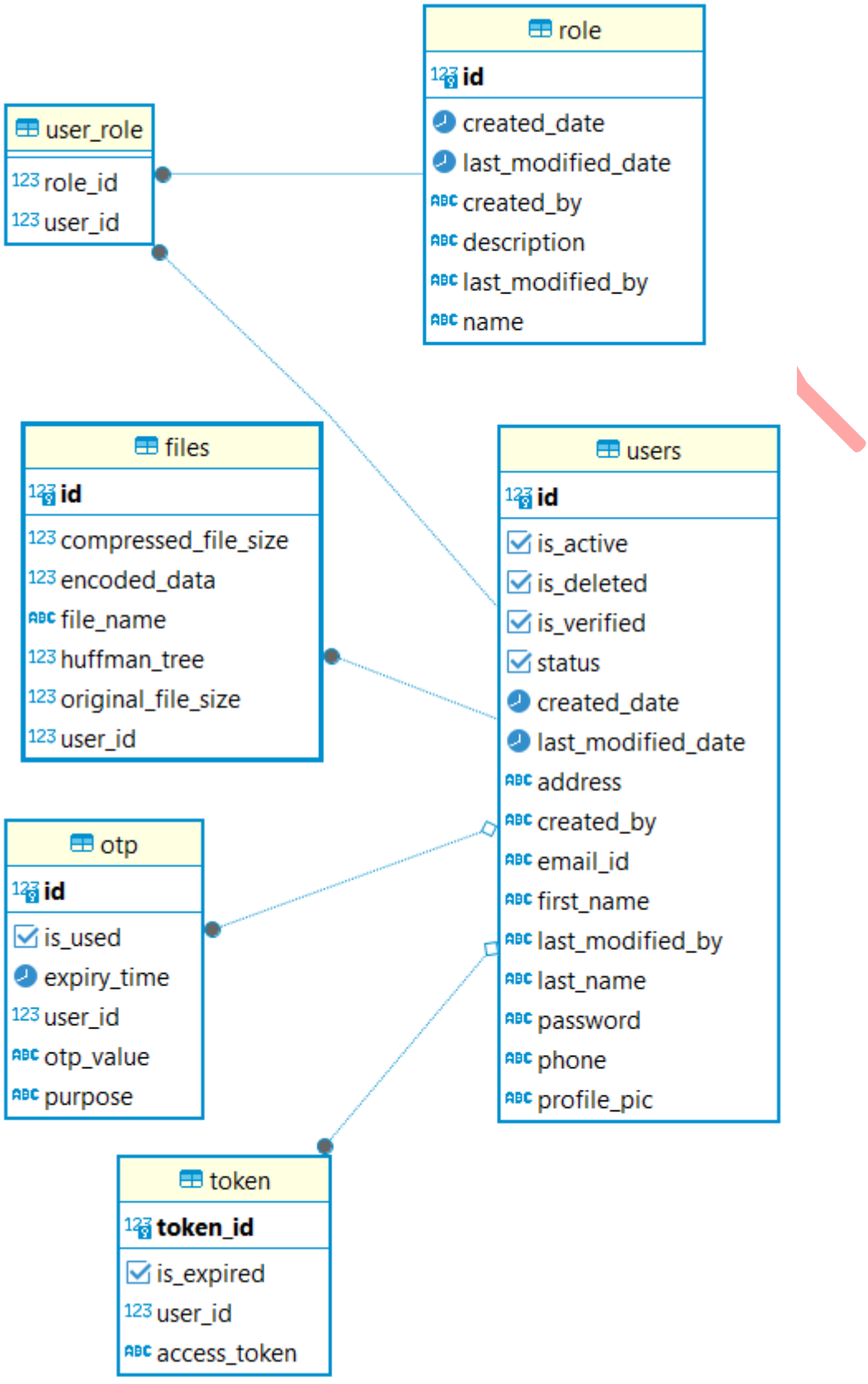


Figure 3. 7 Database Schema Diagram of Project Archival

3.3 Algorithm Details

Huffman Encoding is a lossless data compression algorithm, which is used to reduce the size of data without losing any information. It assigns binary code to characters that occur more frequently and longer codes to those that occur less frequently in the input data. This technique is widely used in applications like file compression (ZIP files), multimedia formats (MP3) and more.

The Huffman Encoding algorithm mainly performs two tasks:

- Creates a binary tree known as the Huffman Tree based on the frequency of characters in the input.
- Generates binary codes for each character such that the most frequent characters have the shortest codes.

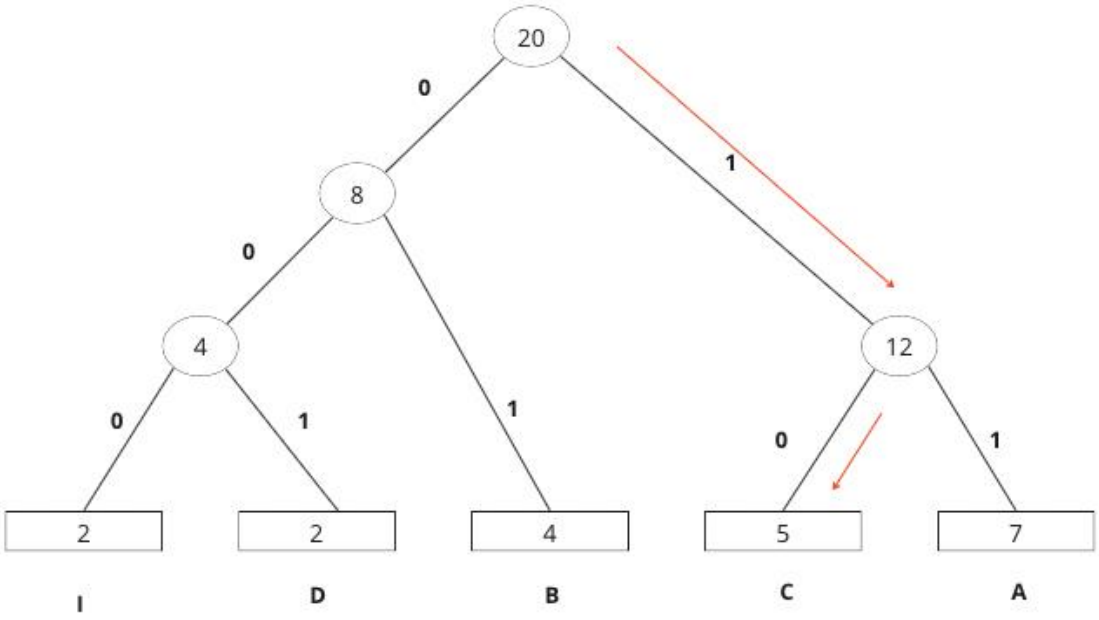


Figure 3. 8 Huffman Encoding Algorithm

The working of the Huffman Encoding algorithm is explained in the below steps:

Step-1: Count the frequency of each character in the input data.

Step-2: Create a priority queue (min heap) and insert all characters along with their frequencies as leaf nodes.

Step-3: While there is more than one node in the queue:

- Remove the two nodes of the lowest frequency from the queue.

- Create a new node with frequency equal to the sum of the two nodes.
- Set the two nodes as left and right children of the new node.
- Insert the new node back into the priority queue.

Step-4: Repeat Step-3 until the priority queue contains only one node. This final node becomes the root of the Huffman Tree.

Step-5: Traverse the Huffman Tree to assign binary codes to each character.

- Assign 0 for left edge and 1 for the right edge (or vice versa)
- The code for a character is the path from the root to its leaf node.

Step-6: Encode the input data using the generated Huffman codes.

Step-7: The compressed output consists of:

- The encoded data.
- The structure of the Huffman Tree, required for decompression.

Step-8: The model is ready for compression and decompression.

PseudoCode for Huffman Algorithm:

```
@Service
class HuffmanCodingService {
   Map<Byte, Integer> calculateHuffmanLengths(List<Integer> weights, int n) {
      // Initialize priority queue for Huffman tree
      PriorityQueue<Node> queue = new PriorityQueue<>((a, b) -> a.weight - b.weight);
      for (int i = 0; i < n; i++) {
         Node leaf = new Node(weights.get(i), (byte) i);
         queue.add(leaf);
      }
      // Build Huffman tree
      for (int i = 0; i < n - 1; i++) {
         Node left = queue.poll();
         Node right = queue.poll();
         Node parent = new Node(left.weight + right.weight, null);
         parent.left = left;
         parent.right = right;
         queue.add(parent);
      }
      // Compute codeword lengths
```

```
        Map<Byte, Integer> lengths = new HashMap<>();
        traverseTree(queue.peek(), 0, lengths);
        return lengths;
    }

    void traverseTree(Node node, int depth, Map<Byte, Integer> lengths) {
        if (node.isLeaf()) {
            lengths.put(node.symbol, depth);
        } else {
            if (node.left != null) traverseTree(node.left, depth + 1, lengths);
            if (node.right != null) traverseTree(node.right, depth + 1, lengths);
        }
    }

    class Node {
        int weight;
        Byte symbol;
        Node left, right;
        Node(int weight, Byte symbol) {
            this.weight = weight;
            this.symbol = symbol;
        }
        boolean isLeaf() { return symbol != null; }
    }
}
```

Chapter 4: Implementation and Testing

4.1 Implementation

During the implementation phase of the Project Archival, the following tools and technologies were utilized to develop the project and prepare documentation.

4.1.1 Tools Used

Category	Tools and Description
CASE Tools	Draw.io for creating diagrams (Waterfall Model, ER, DFDs and so on)
Frontend	Angular for SPA development, Bootstrap for responsive UI.
Backend	Spring Boot for RESTful APIs and business logic, PostgreSQL for data persistence.
Testing	JUnit for unit testing, Postman for API testing.
Development Environment	IntelliJ IDEA and Visual Studio for coding, Git for version control.

4.2 Testing

4.2.1 Test Cases for Unit Testing

Table 4. 1 Test Case for Login and Signup

S.N	Test Case	Test Input	Expected Result	Outcome	Result
1	Sign up/Registration with valid details	User name: Pasang Sherpa Email:pasang@yopmail.com Password: Pasang@123 Address: Kapan Phone no:9865452154 Profile: image	New user is created, and data is stored in database with provided input.	As expected	PASS

2	Registration with invalid details	If there is any Empty input fields	Toast message pops up related to error message.	As expected	PASS
3	Login with valid credentials	Email: pasang@yopmail.com Password: Pasang@123	If user is verified, user is logged in successfully redirect to there specific Dashboard.	As expected	PASS
4	Login with invalid credentials	Email: ram Password: ram@123	Toast error message pops up saying "Invalid username or password".	As expected	PASS

Table 4. 2 Test Case for File Compression

S.N	Test Case	Test Input	Expected Result	Outcome	Result
1	When user compress the file or folder	File: something.txt Description: This is the text data	File Compressed Sucessfully meassage displayed	As expected	PASS
2	When user download the compressed file	X	File with txt or zip extension is downloaded	As expected	PASS

4.2.2 Test Cases for System Testing

System testing evaluated the Project Archival System as a complete application to ensure it met its objectives as a reliable archival solution. Tests confirmed that users could upload files, select compression methods (Huffman or ZIP Deflate), and download compressed files securely without errors, maintaining data integrity. The system's responsiveness was verified across devices (desktop, tablet, mobile), ensuring a user-friendly interface and seamless navigation. Registered user features, such as archive management and usage statistics, were tested to confirm correct folder organization and accurate display of compression history. Admin functionalities were validated by ensuring activity logs accurately reflected system usage and user management actions (e.g., suspending accounts) were applied correctly. Stress testing with 100 concurrent uploads and downloads confirmed system reliability under load. These tests ensured the application functioned as a robust, user-friendly solution for academic project archival.

Chapter 5: Conclusion and Future Recommendations

5.1 Lesson Learnt/ Outcome

After the completion of this project, users can get access to their compress projects, files etc. They can compress their files, documents, projects and get access to it anytime and anywhere.

5.2 Conclusion

This Project Archival System stands out as an intuitive solution for compressing and organizing files and folders of a project using the Huffman and ZIP Deflate algorithms. The system's file size reduction capability supports storage efficiency and optimal space utilization. Users can select from a list of compression methods available to them at the file or project folder level. The system's features enable download of compressed files showing a secure download option, real-time compression analysis, and compression ratio visualization. These features enhance usability and system transparency. In addition to the shared features, registered users gain additional functions of advanced archive management, detailed usage statistics, secured storage, and administrators gain the ability to track system activities and user management. Personally, I believe this project enhances practical file management with foundational data compression techniques for utility and educational benefits.

5.3 Future Recommendation

In near future we are going to implement the following criteria:

- User Activity Logs and History Tracking
- Terminal based Upload system

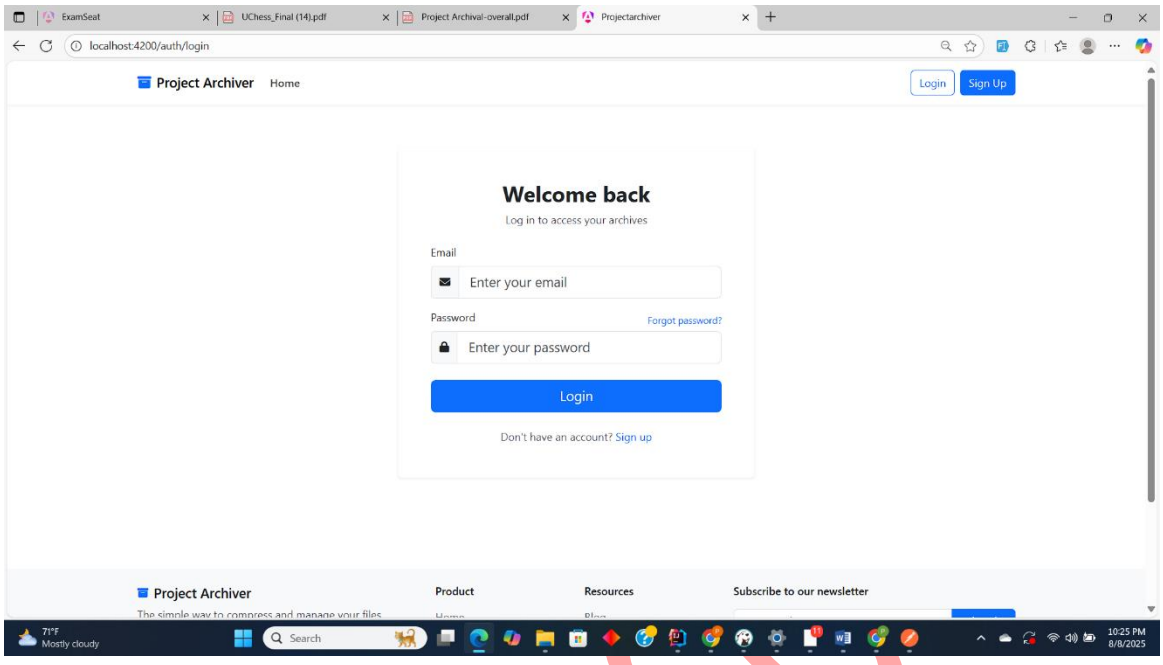
References

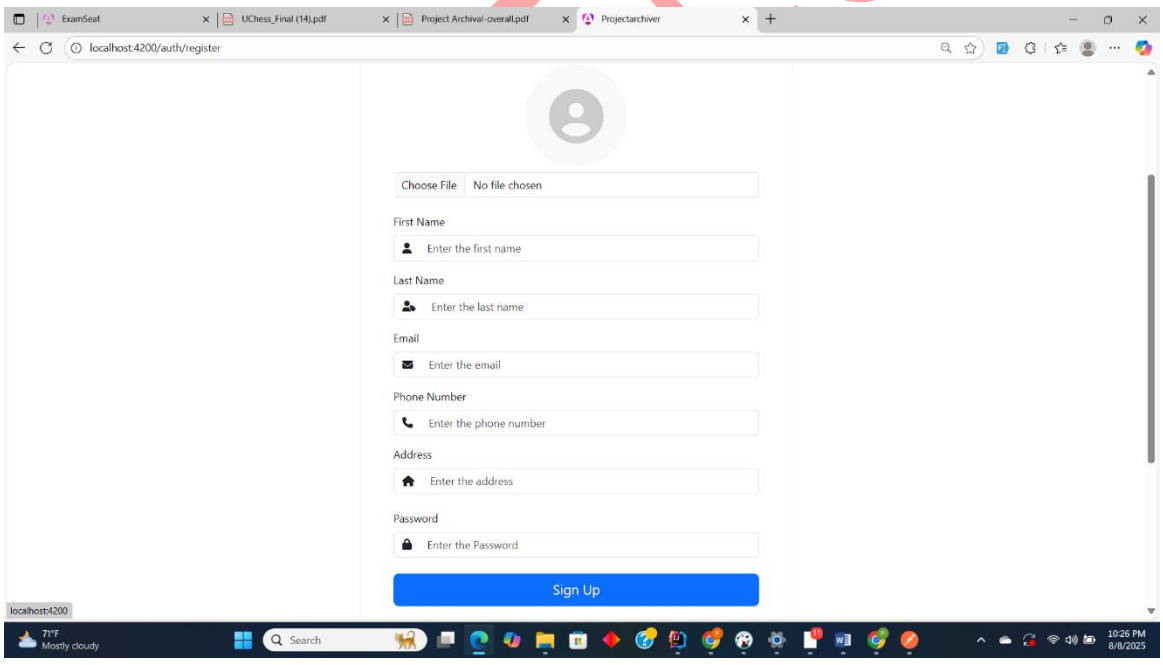
[1] D. Harnik, E. Khaitzin, D. Sotnikov, and S. Taharlev, "A fast implementation of Deflate," in *Proc. 2014 Data Compression Conf.*, Mar. 2014, pp. 223–232, doi: 10.1109/DCC.2014.9.

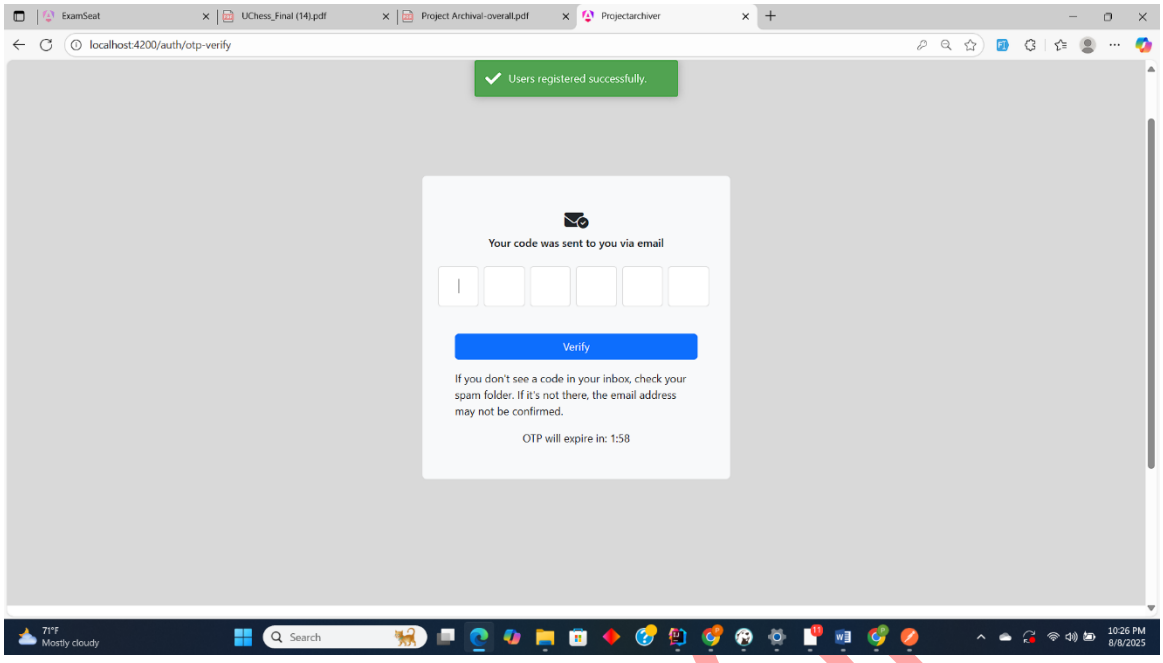
[2] A. Moffat, "Huffman coding," *ACM Comput. Surv.*, vol. 52, no. 4, pp. 1–35, Jun. 2019, doi: 10.1145/3342551.

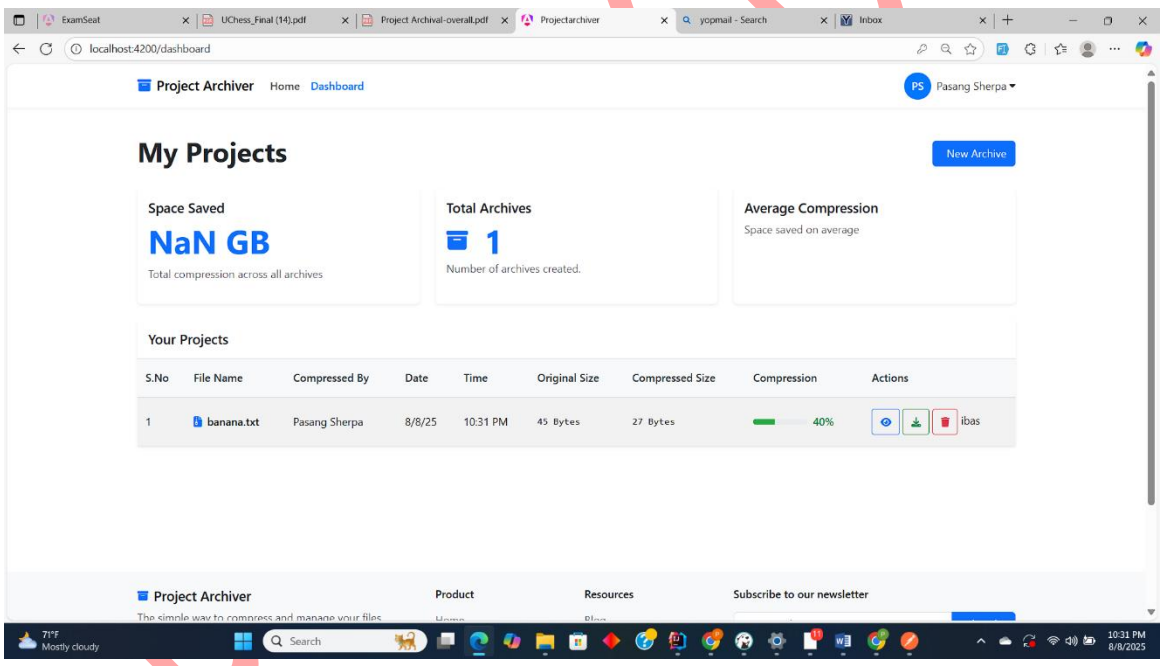
[3] P. Deutsch, "DEFLATE Compressed Data Format Specification version 1.3," *Network Working Group, RFC 1951*, May 1996. [Online]. Available: https://tools.ietf.org/html/rfc1951. [Accessed: Aug. 8, 2025].

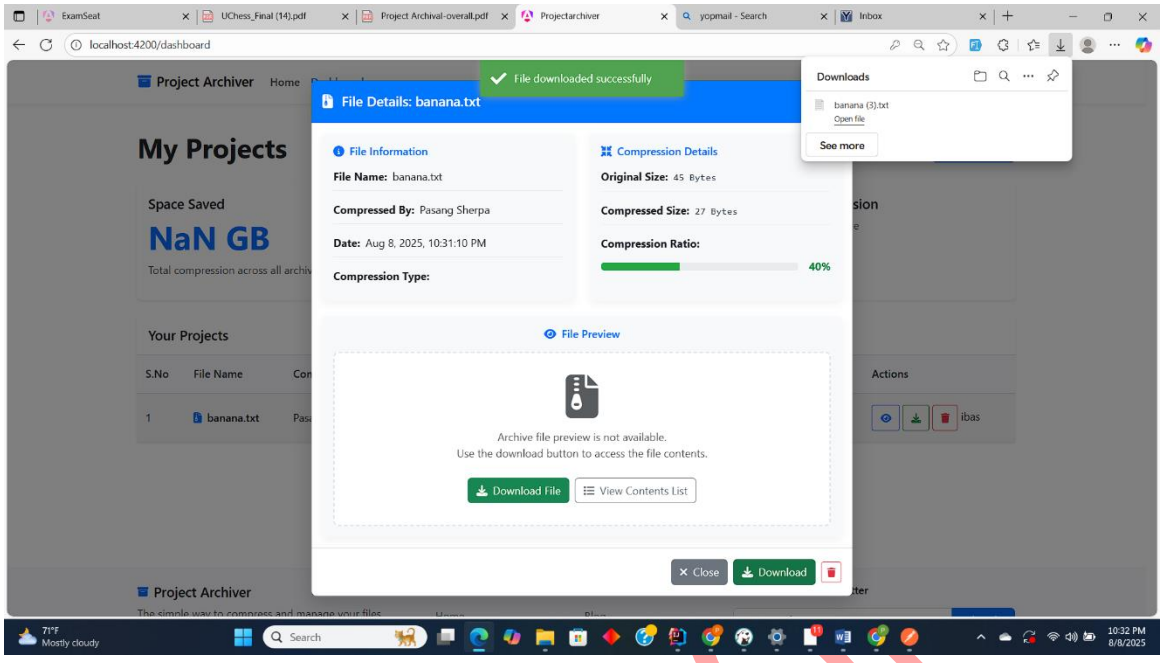
Appendices

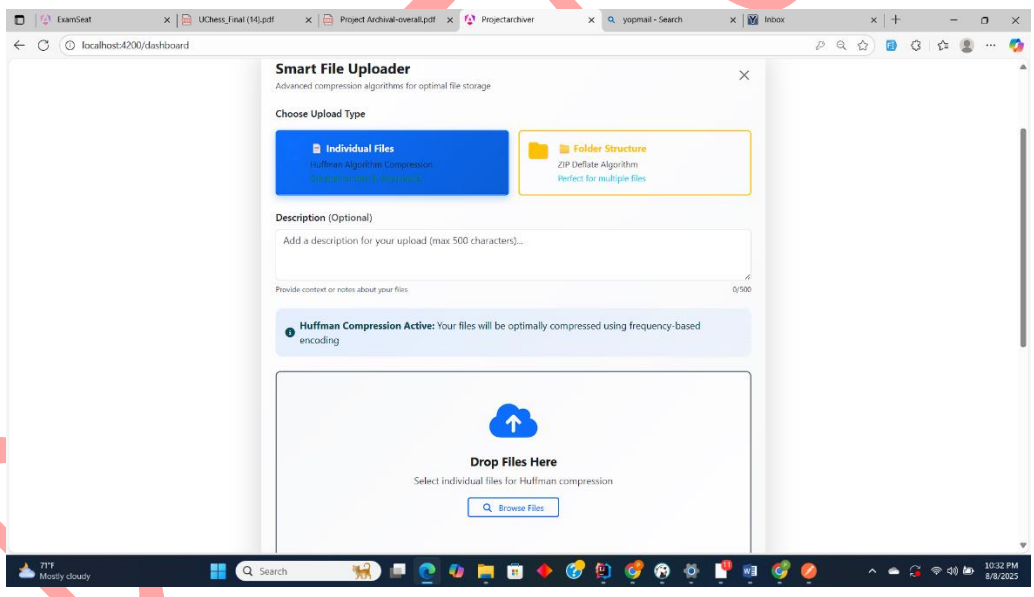

Project Archiver
Home
Login
Sign Up
Welcome back
Log in to access your archives
Email
Enter your email
Password
Forgot password?
Enter your password
Login
Don't have an account? Sign up
Project Archiver
Product
Resources
Subscribe to our newsletter

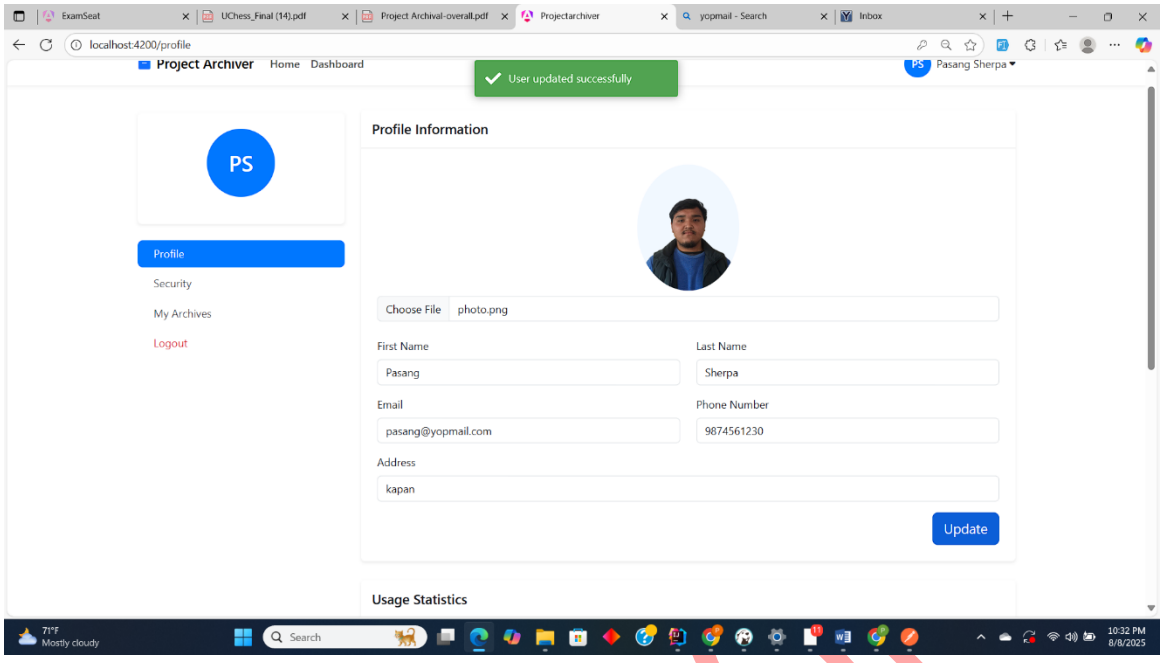

Choose File
No file chosen
First Name
Enter the first name
Last Name
Enter the last name
Email
Enter the email
Phone Number
Enter the phone number
Address
Enter the address
Password
Enter the Password
Sign Up


ExamSeat
UChess_Final (14).pdf
Project Archival-overall.pdf
Projectarchiver
localhost:4200/auth/otp-verify
Users registered successfully.
Your code was sent to you via email
Verify
If you don't see a code in your inbox, check your spam folder. If it's not there, the email address may not be confirmed.
OTP will expire in: 1:58
Search
10:26 PM
8/8/2025


ExamSeat
UChess_Final (14).pdf
Project Archival-overall.pdf
Projectarchiver
yopmail - Search
Inbox
localhost:4200/dashboard
Project Archiver
Home
Dashboard
Pasang Sherpa
My Projects
New Archive
Space Saved
NaN GB
Total compression across all archives
Total Archives
1
Number of archives created.
Average Compression
Space saved on average
Your Projects
S.No
File Name
Compressed By
Date
Time
Original Size
Compressed Size
Compression
Actions
1
banana.txt
Pasang Sherpa
8/8/25
10:31 PM
45 Bytes
27 Bytes
40%
ibas
Project Archiver
Product
Resources
Subscribe to our newsletter
Search
10:31 PM
8/8/2025


localhost:4200/dashboard
Project Archiver
Home
My Projects
Space Saved
NaN GB
Total compression across all archi
Your Projects
S.No
File Name
Actions
1
banana.txt
File downloaded successfully
File Details: banana.txt
File Information
File Name: banana.txt
Compressed By: Pasang Sherpa
Date: Aug 8, 2025, 10:31:10 PM
Compression Type:
Compression Details
Original Size: 45 Bytes
Compressed Size: 27 Bytes
Compression Ratio:
40%
File Preview
Archive file preview is not available.
Use the download button to access the file contents.
Download File
View Contents List
Close
Download
Downloads
banana (3).txt
Open file
See more
Project Archiver
Search


localhost:4200/dashboard
Smart File Uploader
Advanced compression algorithms for optimal file storage
Choose Upload Type
Individual Files
Huffman Algorithm Compression
Folder Structure
ZIP Deflate Algorithm
Perfect for multiple files
Description (Optional)
Add a description for your upload (max 500 characters)...
Provide context or notes about your files
0/500
Huffman Compression Active: Your files will be optimally compressed using frequency-based encoding
Drop Files Here
Select individual files for Huffman compression
Browse Files
Search


ExamSeat
UChess_Final (14).pdf
Project Archival-overall.pdf
Projectarchiver
yopmail - Search
Inbox
localhost:4200/profile
Project Archiver
Home
Dashboard
User updated successfully
Pasang Sherpa
PS
Profile
Security
My Archives
Logout
Profile Information
Choose File
photo.png
First Name
Pasang
Last Name
Sherpa
Email
pasang@yopmail.com
Phone Number
9874561230
Address
kapan
Update
Usage Statistics
71°F
Mostly cloudy
Search
10:32 PM
8/8/2025